Functional AI/ML Proof of Concept

LeadIQ implementation guide, observed evidence and demo runbook

Prepared for Cynbit Technologies

Prepared by Surya Prakash Polkam • 20 October 2026

Assessment deliverable 02 | Confidential submission draft

Functional AI/ML Proof of Concept

LeadIQ implementation guide, observed evidence and demo runbook

Prepared for Cynbit Technologies • Prepared by Surya Prakash Polkam • 20 October 2026

Assessment deliverable 02 | Confidential submission draft

Executive summary

LeadIQ is a local proof of concept that turns free-text project inquiries into structured fields, identifies missing information, produces a readiness category and follow-up draft, and saves analysis records. The implementation combines Gemini through an OpenAI-compatible API interface, Pydantic validation, scikit-learn, Streamlit, FastAPI and SQLAlchemy.

1. Feature inventory and evidence status

Capability	Implementation	Observed status
LLM extraction	Gemini via OpenAI-compatible SDK interface; rule-based fallback when unavailable.	LLM mode displayed in captured dashboard screenshot.
Readiness classification	TF-IDF + Logistic Regression on synthetic examples, with rule guardrails.	Prediction function returned a category; quality is weak on current evaluation.
Dashboard	Streamlit UI showing fields, missing information, next step and reply draft.	Actual running UI screenshots included below.
Persistence	SQLAlchemy with SQLite default and MySQL configuration option.	Saved record #2 confirmation captured.
REST API	FastAPI endpoints /health, /analyze, /analyses.	User reported successful local /analyze smoke test; no API screenshot captured.
Tests	pytest tests for validation and readiness behavior.	6 passed in reported run; pytest cache permission warning.

2. Evidence: running dashboard

The following figures are screenshots supplied during the implementation session. They are evidence of the observed local application state, not mockups.

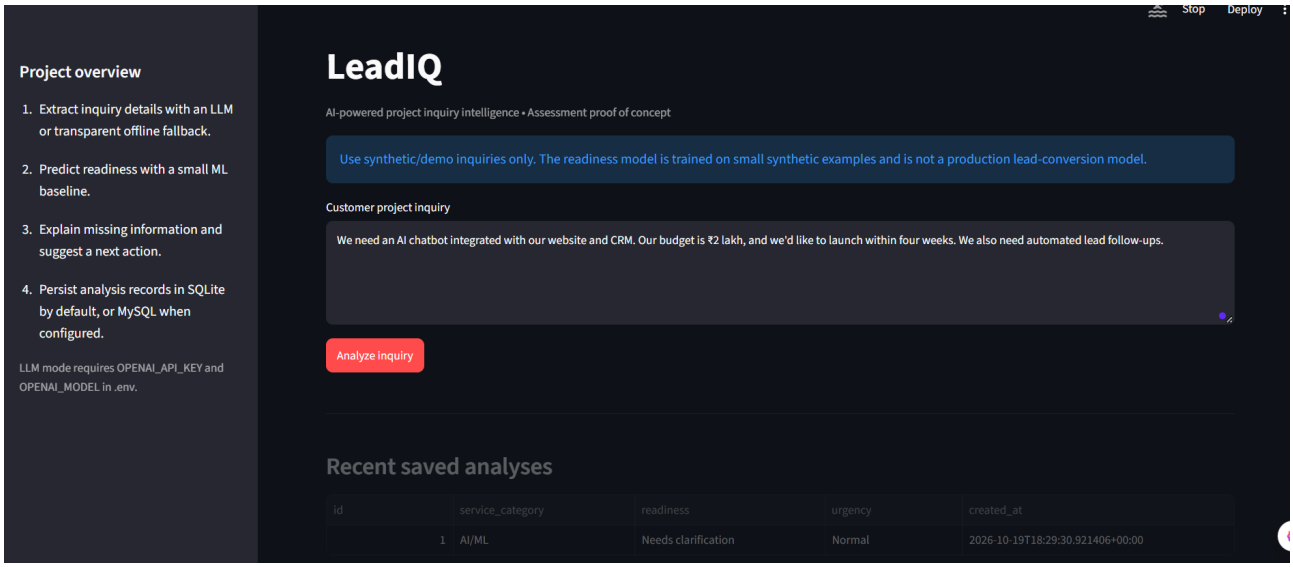


Figure 1. Streamlit LeadIQ dashboard with sample inquiry input.

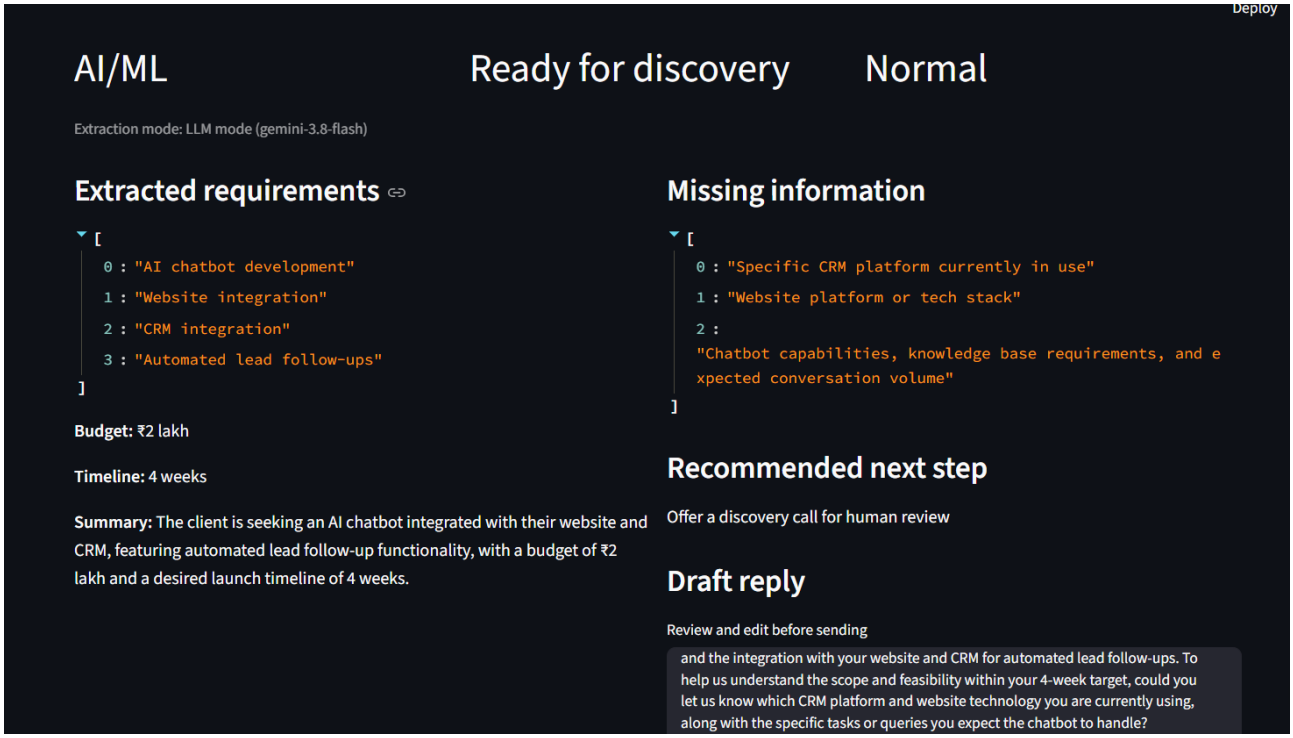


Figure 2. Gemini LLM mode and structured analysis results.

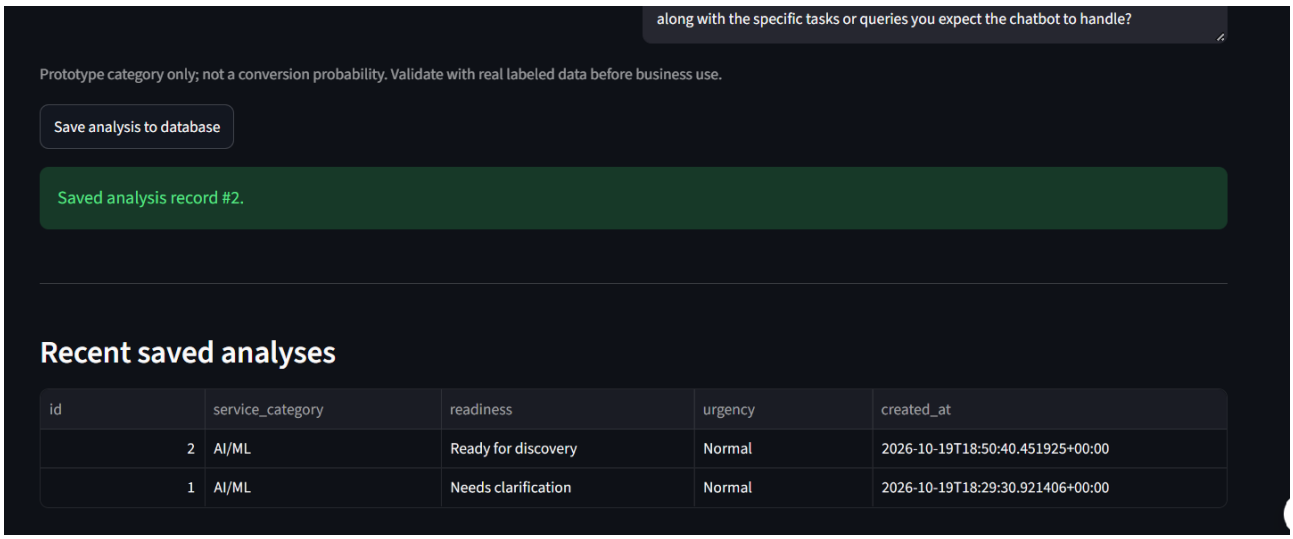


Figure 3. Database save confirmation and recent saved analyses.

3. Additional inquiry scenarios

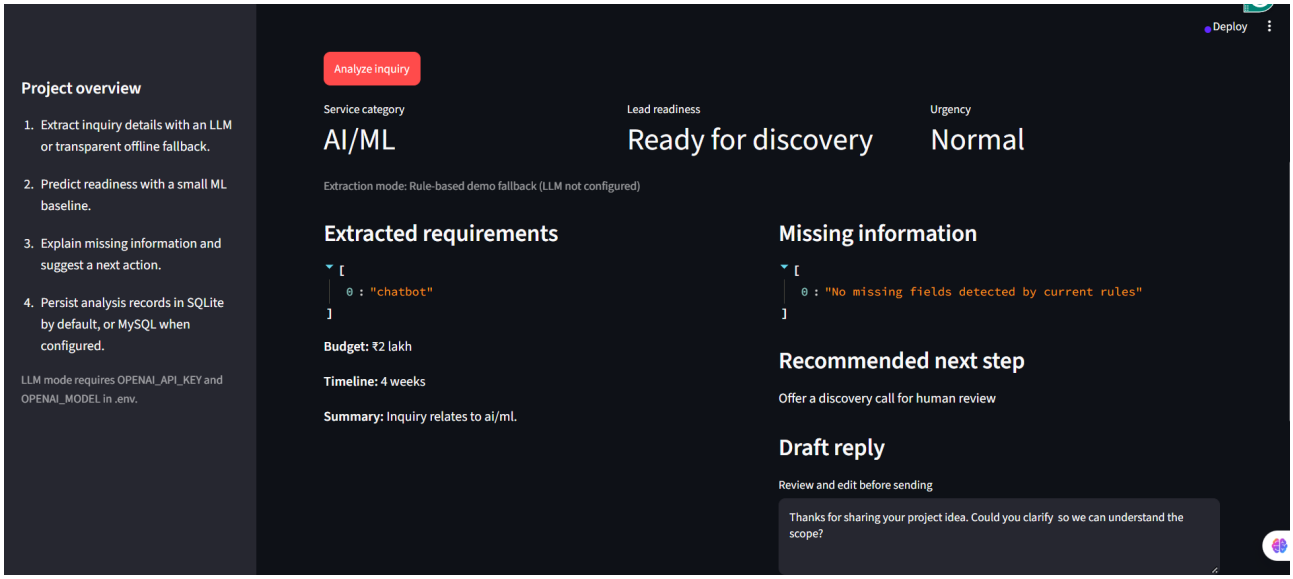


Figure 4. Completed inquiry scenario showing readiness and extracted fields.

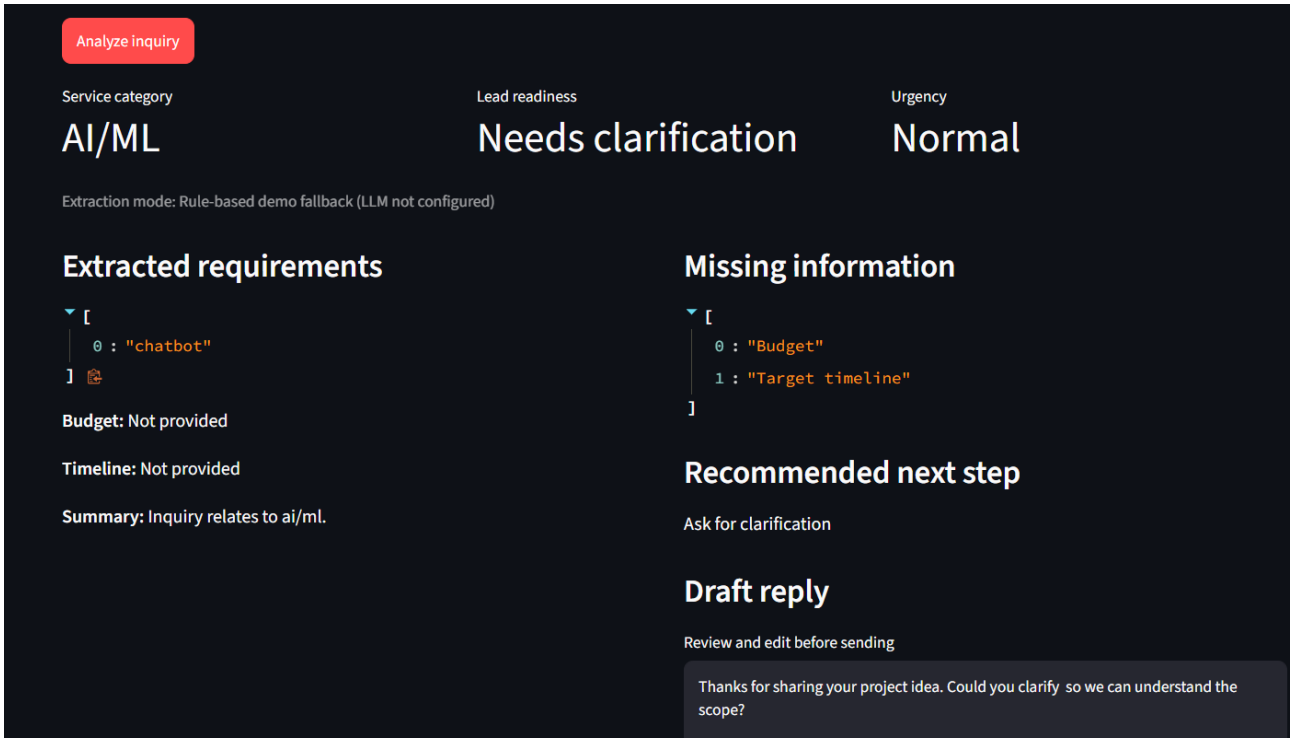


Figure 5. Incomplete inquiry scenario showing a clarification-oriented result.

4. Local setup

Prerequisites: Python 3.10+ recommended, project source folder, internet access and a Gemini API key if using LLM mode.

```
cd C:\Users\surya\Downloads\LeadIQ
python -m pip install -r requirements.txt
Copy-Item .env.example .env
python -m streamlit run app.py
```

Set OPENAI_API_KEY to the Gemini API key and OPENAI_MODEL to a model available to your account. The environment variable name reflects the SDK compatibility interface; it does not mean the key should be an OpenAI key. Never include the real .env file in the submission.

5. Run the API

Open a second PowerShell window:

```
cd C:\Users\surya\Downloads\LeadIQ
python -m uvicorn api:app --reload
```

Open http://127.0.0.1:8000/docs. Test GET /health, then POST /analyze using a synthetic inquiry. Keep the terminal running while testing.

6. Example API request

{
"message": "We need an AI chatbot integrated with our website and CRM. Our budget is INR 2 lakh, and we want to launch within four weeks."
}

7. Demonstration runbook

– Show the dashboard overview and synthetic-data warning.

– Submit a complete sample inquiry and explain the extracted category, requirements, budget and timeline.

– Point out the missing-information section, next action and draft reply.

– Submit an incomplete inquiry and demonstrate the clarification workflow.

– Save the analysis; show the confirmation and recent records.

– Start FastAPI in a second terminal; test /health and /analyze through /docs.

– Run the automated tests and show the ML evaluation result, explaining the small synthetic dataset.

8. Test evidence

Check	Observed result	Limit
Gemini mode	Dashboard showed LLM mode (gemini-3.8-flash).	Model availability may vary by account/time.
Database save	Saved analysis record #2 shown in UI.	Local database smoke test, not concurrent/production test.
FastAPI	Uvicorn startup and /analyze response reported successful.	No screenshot of API response captured in the evidence set.
pytest	6 tests passed.	Functional behavior only; not model-quality proof.
ML cross-validation	Accuracy 0.50; macro F1 0.41; 16 synthetic samples.	Exploratory, tiny dataset; no real-world inference.

9. Known limitations and safe-use rules

• Readiness labels are not conversion probabilities or production lead scores.

• LLM fields and drafts may be wrong; review before using them.

• No live CRM integration, external deployment, authentication or production monitoring is evidenced.

• Use synthetic inquiries for the assessment demo unless real-data processing is approved.

• SQLite is for local demo use; production needs backups, access controls, retention policy and migration management.

• Never commit or share API keys; revoke any key exposed in code or screenshots.

Submission contents

This deliverable is accompanied by the source package, evaluation script, test file and the five evidence screenshots. The API smoke test was reported as successful, but an API-browser screenshot should be captured locally if the client specifically requires visual proof of Swagger testing.